[redacted]

本科生毕业设计

宇宙超级无敌 XXX 系统的设计与实现

院　　系	计算机科学与技术
专业班级	[redacted]
姓　　名	[redacted]
学　　号	[redacted]
指导教师	[redacted]

2020 年 06 月 10 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　年　月　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保密口，在　　年解密后适用本授权书

2、不保密口　。

（请在以上相应方框内打“√”）

作者签名：　　年　月　日

导师签名：　　年　月　日

摘 要

理想的存储体系结构应该提供安全性、跨平台数据共享、高性能、以及可扩展性，然而当前广泛使用的三种存储结构（DAS，SAN 和 NAS）都不能兼顾所有这些需求。因此，工业界和学术界正在努力改变目前的存储技术，使存储设备从非智能的、外部管理发展成为智能的、自管理的设备，并且能够感知设备所服务的应用。基于对象的存储（Object-Based Storage）被认为是该问题的解决之道。

以 SCSI OSD 协议为基础，建立属性的两种传递机制。一种是调用专门的应用程序接口直接提供属性信息，存储设备根据属性直接做出存储策略选择。另一种是应用程序提供文件系统层的信息，设备内部根据文件的属性做出相关预测，并用于策略选择。比较而言，前者容易实现，而后者要求新的 OSD 文件系统，好处是不需要修改应用程序。同时还给出了三个属性页的定义作为 OSD 标准扩展提议。设计实现了符合 T10 OSD 标准协议的原型系统（iSCSI-OSD-RAID）。利用 iSCSI 协议作为 OSD 命令的传输层，加快了系统的实现过程。同时以一个正在进行的 RAID 控制器项目为基础，增加了新的对象存储管理模块，最终实现了 OSD 接口的磁盘阵列，并验证了属性控制的数据放置策略。实验测试结果表明….

关键词：对象存储；磁盘阵列；存储策略；数据放置

摘要撰写说明，阅后焚毁！！

中文摘要是对论文内容的高度概括，应用精练的语言概述论文的主要研究内容、目的意义、设计过程、实验手段及取得的成果等。摘要一般分为 2~3 段，第一段简要介绍背景知识，尽量简洁，切中要害，不要说些任何人都知道的无信息量的语句；后面可以用一到两段介绍毕业设计工作，主要体现自己的工作，摘要不要太长，但主要工作部分应该比第一段背景知识长，不要本末倒置。

摘要中不得出现“本文共有 X 章，第一章…，第二章…”之类的表述。摘要严禁出现“本文”，“我”，“我们”这样的第一人称主语，尽量采用动宾结构，比

如设计了….实现了…利用了….技术…..进行了….实验…..实验结果表明……，如果需要设置主语可以用文中涉及的系统或研究机制等。

摘要不要超过一页，关键词与摘要在同一页，数量 3～7 个。中文关键词须用汉字，尽量不使用英文单词或其缩写，例如“DBMS”不能作为中文关键词，必须用对应的中文表述：“数据库管理系统”。关键词之间用逗号分隔，最后一个关键词后不用标点。关键词应该具体，不得用过于泛化的词做关键词，如“音乐、视频”，形容词不能作关键词。

Abstract

Failure resilience is of pivotal importance in practical network function virtualization (NFV) systems, but has been mostly absent in the existing ones. The absence is mainly due to the challenge of patching source code of the existing NF software for extracting important NF states, a necessary step toward flow migration and replication to provide failure tolerance.

This paper proposes NFActor, a novel NFV system that uses the actor programming model to provide transparent resilience, high scalability and low overhead in network flow processing. In NFActor, a set of efficient APIs are provided for constructing NFs, with inherent support for scalability and resilience. A per-flow management principle is advocated, different from the existing practice, which provides dedicated micro service chain services for individual flows, enabling decentralized flow migration and scalable flow replication. We implement NFActor and show that it achieves good scalability, prompt flow migration and failure recovery with large numbers of concurrent flows. We also show that NFActor can enable applications such as live NF update and correct MPTCP subflow processing, which cannot be efficiently achieved in previous systems.

Keywords: Network Function Virtualization (NFV), Failure resilience, Actor model, Flow migration, Service Chain

撰写说明，阅后删除！！！

英文摘要在中文摘要定稿后再开始撰写，否则会多次修改，不要过于依赖百度翻译等工具。

Abstract 是论文的英文摘要，一般对照中文摘要翻译，要求另起一页。Keywords 对照中文关键词翻译，与英文摘要在同一页，以“Keywords：”另行顶左开始。Keywords 之间用“,”隔开，最后一个 Keyword 后不用加任何标点符号。

目 录

摘 要 ..... **I**
Abstract ..... **III**
1 绪 论 **1**
1.1 课题背景 1
1.2 国内外研究现状 2
1.3 研究目的和主要内容 3
1.4 论文结构 4
1.5 课题来源 4
2 方案论证（或具体背景技术概述） **5**
2.1 系统需求分析 5
2.2 系统可行性分析 6
2.3 开发工具分析及选择 6
2.4 关键技术分析 6
2.5 基本方案制定 7
2.6 本章小结 7
3 XXX 系统设计 **8**
3.1 功能需求 8
3.2 系统总体设计 9
3.3 功能模块设计 13
3.4 本章小结 16
4 XXX 系统实现 **17**
4.1 过滤器实现 17
4.2 属性管理模块实现 18
4.3 数据迁移模块实现 19
4.4 本章小结 22
5 性能测试与分析 **23**
5.1 测试环境 23
5.2 功能测试 23

5.3 系统界面.....23
5.4 性能测试.....23
5.5 本章小结.....25
6 总结与展望.....26
致 谢.....27
7 毕业设计模板基本框架.....28
7.1 封面.....28
7.2 原创性声明页.....29
7.3 摘要.....29
7.4 目录.....29
7.5 参考文献.....30
7.6 附录.....30
7.7 毕业设计任务书.....30
7.8 成绩评定页.....31
8 毕业设计撰写要求.....32
8.1 图的格式.....32
8.2 表的格式要求.....36
8.3 公式.....38
8.4 流程图.....39
8.5 常见格式问题.....39
参考文献.....41
附录：大学期间发表或提交的论文.....44

撰写说明，阅后删除！！！

目录不会自动更新，排版变动后必须更新目录，更新目录方式：鼠标点击目录，按 F9 键或者右键更新域，选择更新整个目录即可。

目录仅包含一级标题和二级标题和标题样式，目录是全论文的纲要。中文摘要、Abstract、论文正文的各级标题（不包括第三级）、致谢、参考文献、附录等都应编入目录，标注其页码对照关系，但目录本身不出现在其中。中文摘要、Abstract、目录等使用希腊数字“I、II、…”编连续页码；论文正文、致谢、参考

文献、附录等使用“1，2，3，… ”编连续页码。

1 绪 论

本章我们首先介绍了当前存储系统面临的挑战和技术发展趋势，然后分析了对象存储技术的产生及发展现状，介绍了国内外在存储策略和智能存储领域的相关研究工作，并对本文的主要研究内容及工作意义作了具体说明。(除总结那一章以外，每章都加一段引言)

1.1 课题背景

绪论主要介绍本文的选题背景：说明本设计课题的来源、目的、意义、应解决的主要问题及应达到的技术要求；简述本课题在国内外发展概况及存在的问题，本设计的指导思想等。（请在开题报告中完成第一章绪论部分的内容）

1.1.1 研究背景和趋势

网络功能虚拟化（NFV）主要是将网络功能从专用的硬件设备中迁移到通用的服务器上，然后让这些网络功能以虚拟的应用的形式运行在通用的服务器上。在 NFV 的帮助下，网络运营商可以通过启动虚拟设备运行网络功能的方式快速提供网络服务。一个网络服务一般由一个序列的网络功能组成用来进行流量处理，一般也称之为服务链。比如“防火墙->入侵检测系统->负载均衡器”这就是一条服务链。

近几年也出现了一些 NFV 的管理系统，比如 E2[1], OpenBox[2], CoMb[3], Xomb[4], Stratos[5] 和 OpenNetVM[6]. 他们实现了大量的管理功能，包括动态的网络功能放置，网络功能的扩展，负载均衡等等。但是他们往往都缺少错误恢复功能，也往往会缺少对流量迁移[7][8][9]的有效支持。

1.1.2 提高存储系统性能的途径

错误恢复功能对有状态的网络功能来说非常重要。很多网络功能都会维护一个基于单个流的状态[10]: 入侵检测系统例如 Bro[11]为每一个流保存并更新与协议有关的状态来对可能的攻击进行预警；防火墙[12]会解析 TCP SYN/ACK/FIN 数据报文并为每一个流维护与 TCP 连接相关的状态；负载均衡器为了修改流中报

文的目的地址，会保存每个 flow 标识符与对应地址的映射关系。在网络功能异常挂机的情况下，能够保证正确的流量状态的恢复就显得尤其重要，这样才能保证那些网络功能处理的连接不需要重置。而重置连接这个简单的方法一般并不被大多数的网络供应商所接受[13]。

1.1.3 面临的问题和挑战

高度去中心化流量迁移和备份。基于 actor 框架，NFActor 中的流迁移和备份过程通过去中心化的消息传递自动进行。中央协调器只在流迁移和备份的初始化阶段才会参与其中。此外，NFActor 中的运行时系统使用高速分组 I/O 库 DPDK [12]来检索/发送流分组，和用来进行流量迁移和备份的控制消息。

我们实现了 NFActor 并将其开源。我们的测试实验表明，NFActor 可以很好地实现高速分组处理，在并行迁移超过 100K 流时可以实现零分组丢失，以及在运行时系统故障的情况下可以在几十毫秒内恢复数万个流。除了错误恢复，实验还显示，NFActor 可以支持一些诸如实时 NF 更新，MPTCP 子流正确处理等应用。这些应用程序需要单独的 NF 来启动流迁移，这在现有系统中难以用小的开销实现。而我们的去中心化和快速的流迁移可以很容易支持这些应用。

1.2 国内外研究现状

必须有有足够的文献阅读量，这部分替代原来的文献综述。

1.2.1 网络功能虚拟化（Network Function Virtualization）

自从引入 NFV 概念[19]以来，很多相关的研究已经陆续开展起来：弥合专用硬件和网络功能之间的差距[20][21][22][23]，扩展和管理 NFV 系统[24][25] 流量迁移[7][8][9]，NF 备份[26][27]和流量转发[28]。NF 实例通常作为在标准虚拟机或容器上运行的软件模块而被创建。NFActor 自定义了运行时系统以“一个 actor 一个流”格式来运行 NF，以实现透明的错误恢复。

在现有研究中，ClickOS[22]还引入了模块化设计，简化了 NF 结构; 然而，在设计之后，一些高级的控制功能例如流量迁移，仍不容易被集成到 NF 中。Flurries[29]通过动态地将流量分配给轻量级的 NF 来提出细粒度的单流 NF 处理。类似

的，NFActor 在一个 actor 中实现每个流的微服务链处理，并且着重于基于 actor 模型来提供透明的错误恢复功能。OpenBox[30]合并了多个 VNF 的处理逻辑，从而提高了模块化和处理性能。OpenBox 的想法也可以应用于 NFActor，而 NFActor 则主要关注主流服务链处理。

为了实现流迁移，现有系统[7][8]需要直接修改 NF 软件的核心处理逻辑，主要依靠 SDN 控制器来执行迁移协议，这导致不可忽略的开销。NFActor 使用新颖设计的 NF API 和分布式框架来克服这些问题，在这个框架中流状态非常容易提取，并且只需 3 个请求响应步骤就可以让流在运行时系统之间实现高速迁移。

提供故障恢复的流备份机制通常涉及检查整个过程映像（NF 软件正在运行的位置），以及使用映像创建副本[26] [27]。而这通常需要暂时停止 NF 进程，从而导致丢包。基于 NF 处理逻辑和流状态之间的干净分离，使得 NFActor 能够以轻量级，透明的方式检查备份流的所有状态，在不停止 NF 进程的情况下以最大限度地减少损失和延迟。

1.2.2 Actor 模型简介

CMU 在 NASD 研究的基础上，实现了一个基于对象的原型系统 Ursa Minor，号称“万能的”集群存储系统[9]。没有单独一种数据分布可以适合所有的数据、使所有的负载达到最优性能。而万能的存储系统允许以对象为单位选择最合适的数据分布（包括编码、容错和位置等），满足相应的访问模式、可靠性需求和成本目标。Ursa Minor 的实现包括：存储节点保存所有数据；对象管理器维护对象的元数据；客户端有 API 库为应用封装数据请求，因为客户端与存储节点、对象管理器之间的通信使用自定义的协议；NFS 服务器为那些没有经过修改的客户端提供访问通道。Ursa Minor 协议还支持根据负载的特征（读写比率或者请求大小）在线改变存储节点的配置，使之更加适合负载。

1.3 研究目的和主要内容

在本次课题的研究中，首先我们先构思设计出 actor 的模型，并使用这个模型构建流 actor 的雏形，然后使用 DPDK 来设计运行时系统之间的可靠传输，并使用 GRPC 来构建控制器与运行时系统的通信。然后设计出网络功能的统一编

程接口，提供网络功能核心处理逻辑和状态的分离，最终目的是实现课题要求。

1.4 论文结构

本文的主要内容如下：

第一章我们首先介绍了当前存储系统面临的挑战和技术发展趋势，然后分析了对象存储技术的产生及发展现状，介绍了国内外在存储策略和智能存储领域的相关研究工作，并对本文的主要研究内容及工作意义作了具体说明。

第二章首先介绍几种存储技术、特别是存储区域网（SAN）在安全性、数据共享和设备管理等方面的存在问题。然后详细论述了基于对象的存储如何解决当前的问题。最后讨论了对象存储的实现途径，OSD 标准和 Lustre 文件系统代表了两种发展方向。本文将重点以 OSD 为基础进行有关研究。

第三章以磁盘阵列作为研究对象，描述了属性控制的数据放置策略。我们建立了基于闭环 fork-join 排队网络（CFJQN）的 RAID-5 读写模型，对最优分条单元大小进行了定量分析，并归纳得到可以用来准确描述一个负载的若干属性（包括存储需求和行为）。

第五章比较详细的描述了一个原型系统的设计和实现。iSCSI-OSD-RAID 成功地对传统的 RAID 控制器进行了扩展，增加了支持 iSCSI 和 OSD 协议的访问接口。除了软件方面的工作之外，还实现了基于 Intel IOP321 处理器的控制器硬件，本章对硬件结构和交叉开发环境也进行了介绍。最后还给出了部分测试结果。

第六章总结了所做的工作，并计划了下一步的工作以及展望了基于对象存储的前景。

1.5 课题来源

本课题受国家自然科学基金项目“XXXXXX”（项目编号：60273073）和“973”国家重点基础研究发展项目“XXXX”（项目编号：200418203）资助。课题来源于香港大学工程学院计算机系系统与网络实验组的网络功能虚拟化项目。自选课题可不写。

2 方案论证（或具体背景技术概述）

第二章可以进行项目方案论证或进行具体背景知识的概述，说明设计原理并进行方案选择，阐明为什么要选择这个设计方案，包括各种方案的分析、比较，以及所采用方案的特点

本章名字可变，名字最后可结合自己的课题内容，如果毕业设计是一个大项目的一个模块，本章可以介绍整体项目结构以及相关背景知识，如果毕业设计是科学研讨型，第二章，第三章都可以介绍理论背景知识。

（除总结那一章以外，每章都加一段引言）

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过高、性能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.1 系统需求分析

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过高、性能的下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健的康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自的动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.2 系统可行性分析

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过高代、性能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.3 开发工具分析及选择

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率带过高、性能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.4 关键技术分析

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过的高、性能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统

中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.5 基本方案制定

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过高、性的能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康的指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

2.6 本章小结

（第二章到倒数第二章每章都必须要有一个小结）本章提出了一种可以提高存储系统数据可靠性的技术，即存储设备安全预警及数据主动迁移技术，它包括磁盘数据自愈、磁盘数据主动迁移、阵列级数据自愈及系统级数据主动迁移四个层次。对磁盘数据自愈与数据主动迁移进行了详细描述。

3 XXX 系统设计

前面论述了存储接口从基于“数据块”发展到“对象”的必要性、重要性和可行性。基于对象的接口拥有更强的表达力，其基本手段是通过“属性”描述数据的信息。不同的属性内容服务于不同的目的。这里我们将以磁盘阵列为研究对象，重点是考虑需要获得什么样的负载属性信息，从而为存储对象选择合适的 RAID 级别和分条单元大小。

3.1 功能需求

（1）JGMSA30 型加固磁盘阵列在磁盘存在一部分坏扇区时能够继续工作，当磁盘出现一部分扇区时，阵列并不立即使用新盘替换并重建数据，该磁盘通过自愈可以继续正常工作，为数据迁移提供时间窗口；

（2）能够将磁盘坏扇区数据恢复，根据磁盘所在 RAID 阵列对应的 RAID 算法，通过获取其它成员盘的同一 Stripe 上的数据由，RAID 算法计算恢复出坏扇区数据；

（3）能够检测磁盘健康状况，并在磁盘健康状况不佳时将数据主动迁移到阵列中其它空闲磁盘中，迁移过程不影响正常业务流程，对系统性能影响应该降低到最低限；

（4）能够通过 WEB 界面进行统一监控，用户可通过 WEB 界面对磁盘自愈与数据主动迁移进行相关配置操作，例如可以配置磁盘健康参数、使用数据迁移的目标磁盘替换坏扇区磁盘等；

（5）一般的系统都必须要有需求分析，有功能需求，界面需求，接口需求，性能需求等等。具体参照软件工程，这是系统设计的重要依据，一定要将自己做什么说清楚，如果第二章写了需求这章就不需要了

。。。。

3.2 系统总体设计

3.2.1 系统硬件平台

JGMSA30 型磁盘阵列采用嵌入式技术*[25-27]*实现，其硬件结构主要是在支持 64 位 PCI-X 接口的主板增加相应的芯片构成，例如网卡等相关芯片。将光纤芯片加入到该平台中，以支持光纤访问该阵列，同时还具有 4 块 SATA 芯片，能够连接多达 16 个 SATA 硬盘。为了提高系统计算校验的速度，该系统还加入了 XOR 硬件，使得 RAID5 及其它 RAID 级别计算校验信息的速度大大提高。给出了 JGMSA30 型加固磁盘阵列硬件结构图。**（有硬件系统需要提供硬件环境示意图）**

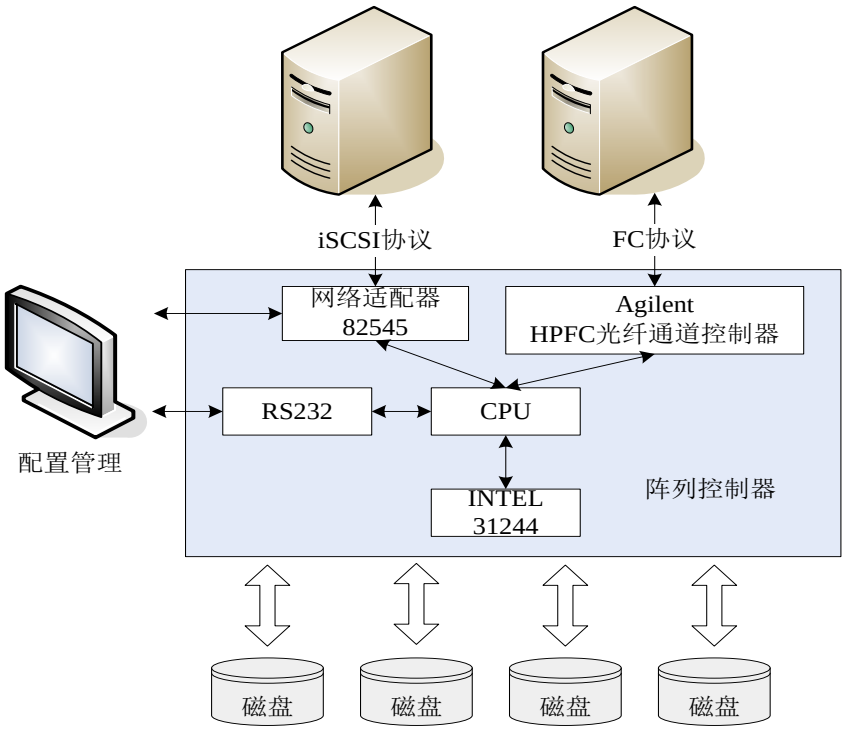

图 3-1 磁盘阵列硬件平台

JGMSA30 型加固磁盘阵列有三种方式与主机交互，分别为网络连接、光纤连接及串口连接，其中光纤是数据通路，即存储数据到阵列系统时，可以通过光纤连接传输数据，网络连接既是数据通路，也是配置信息通路，可以通过 WEB 浏览器配置 RAID 系统，此时通过网络连接交互，而当阵列配置成 ISCSI 访问时，网络连接也是数据通路，串口连接输出该阵列的调试信息，在实际使用时一

般不连接。

3.2.2 系统软件架构

JGMSA30 型加固磁盘阵列控制器上运行嵌入式 Linux 操作系统*[26-29]*，整个系统的软件部分是基于 Linux 系统开发的。为了提高该系统核心程序的可移植性，采用 Linux 内核模块的方式实现。给出了 RAID 系统核心程序的模块划分图，其中 DH 模块是需要设计实现的模块，它位于 RAID 核心程序与 Linux 操作系统提供的读写磁盘接口之间，RAID 核心程序调用 DH 模块提供的读写磁盘接口，DH 模块调用操作系统提供的读写磁盘接口真正读写磁盘，这样 DH 模块就能够过滤 RAID 系统发出的磁盘读写请求，分析请求执行状态，以实现健康状况的监测及对读写出错的扇区进行磁盘数据自愈，并在磁盘健康状况偏离正常时启动数据迁移。（软件整体结构图必须提供，有些系统还需要提供详细的动态运行分析图）

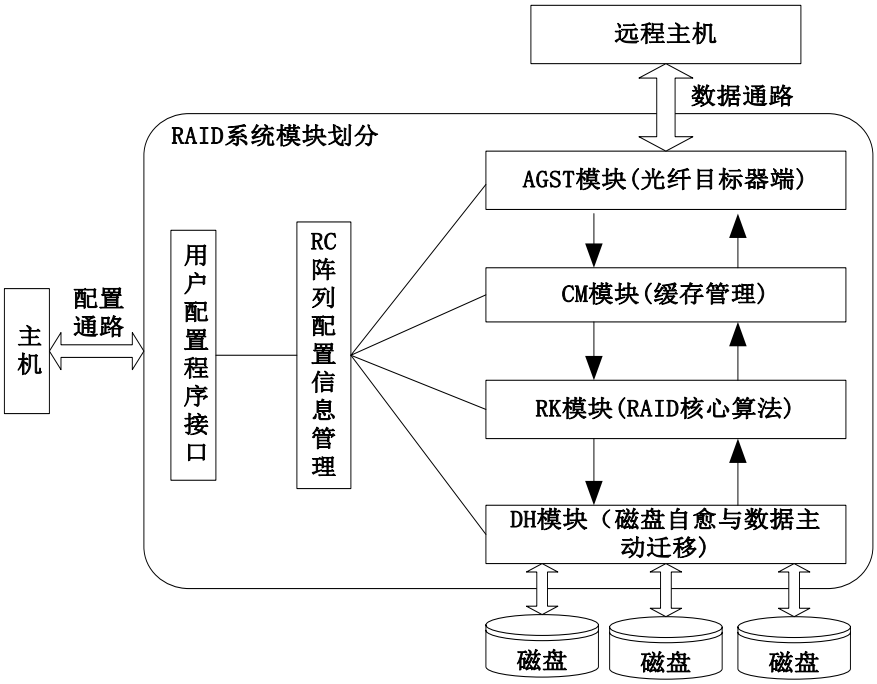

图 3-2 XXX 软件架构

JGMSA30 型加固磁盘阵列系统*[30-35]*软件部分主要分为 6 个模块，分别为 RC 阵列配置信息管理、CM 模块、RAID 核心算法模块、 DH 模块以及用户配置程序接口模块。下面对这几个模块功能做简单介绍。

（1）RC 阵列配置信息管理模块

阵列配置信息管理模块管理整个阵列配置信息，这些信息包括 RAID 系统中所接磁盘信息，以及这些磁盘所属的 RAID 组信息(RAID1、RAID5 等)，还包括每个 RAID 组包含的逻辑卷的详细信息。

（2）AGST 模块

AGST 模块与光纤设备进行交互，负责接收用户发来的读写请求，并传输数据，它是整个 RAID 系统存储数据的窗户。

（3）CM 模块

该模块负责管理 RAID 系统 Cache，这部分用于加速用户访问逻辑卷的速度，将用户经常访问的速度存于 Cache 中，用户访问时，只需要与 AGST 模块及 CM 模块交互，不需要真正访问磁盘，能够提高 I/O 访问速度。

传统存储系统中数据的可靠性通常是采用冗余技术或者备份技术来实现的，如果存储节点出现了如风扇损毁，磁盘温度过高、误码率过高、性能下降等问题，系统通常不会进行主动处理，而是等待设备或者磁盘完全故障后才通过数据重建或热切换到镜像节点的方式保持存储系统的持续工作，这大大增加了数据风险性。本文提出了主动监控的思想，它对存储系统进行及时的控管，定时对系统中的各个存储节点工作温度、节点能耗、数据误码率、传输性能等健康指标进行分析，当健康指标超出正常阈值时，及时产生预警信息，系统随后自动启动数据迁移，将数据迁移到适当的节点或磁盘，这就是存储设备健康预警及数据迁移技术。

3.2.3 开发环境

整个系统的软硬件设计是同步进行的。在自己定制的控制器硬件可以使用之前，我们使用了 IQ80321 评估板作为软件开发平台。该硬件平台是目前开发网络存储设备比较好的选择。(http://www.intel.com/design/IIO/)

操作系统的选择将对系统性能和开发过程产生重要影响。已经移植到 80321 的操作系统包括 Nucleus，MontaVista Linux 和 VxWorks 等。我们选择 Linux 作为嵌入式操作系统。创建嵌入式 Linux 系统的基本任务是在目标板上运行 Linux 内核。参考如**错误!未找到引用源。**所示的“主机/目标机”交叉开发环境，建立一个嵌入式 ARM Linux 系统需要下面 4 个步骤。

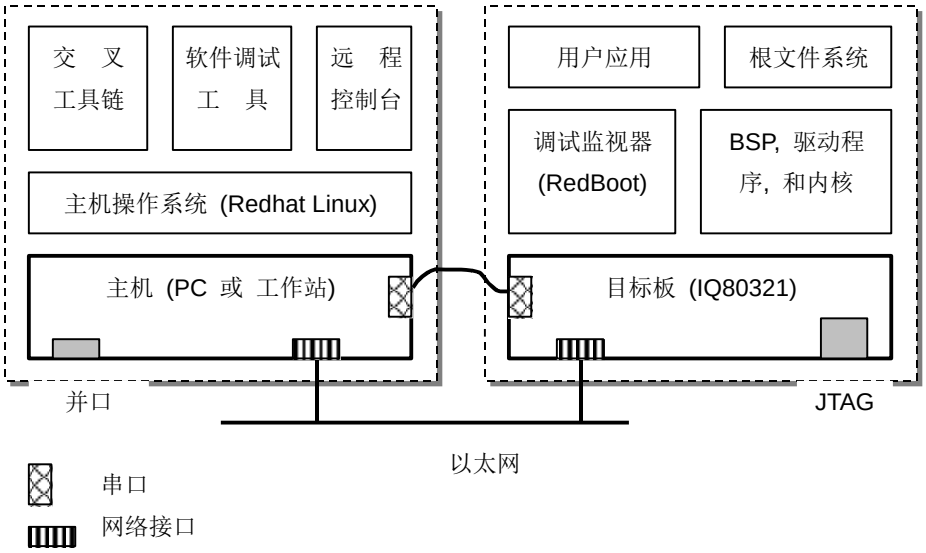

图 3-3 交叉开发环境

1）准备交叉开发工具链

工具链（toolchain）实际上包含许多部件，其中最重要的一个是 gcc 编译器，另一个是 binutils（一组操作二进制文件的工具）。有了这些工具就可以编译内核了，不过其实通常情况下还需要 C 库 glibc，以支持编译更多的应用。

2）编译 ARM Linux 内核

为了编译一个可以在 IOP321 上运行的 Linux 内核，需要为官方 Linux 内核打上一系列补丁。我们为 linux-2.4.21 官方内核源代码打 rmk1，ds0 和 dj9 补丁包，从而得到 linux-2.4.21-rmk1-ds0-dj9 版本内核。然后使用 make 命令即可调用 arm-linux 工具链生成内核映像文件 zImage。后来我们又将内核升级到了 linux-2.6.10-iop1。

3）编译 ARM Linux 内核

3.2.4 功能模块划分

磁盘自愈与数据主动迁移模块提供读写磁盘接口供其它模块调用，然后再调用操作系统提供的读写磁盘接口，需要对磁盘坏扇区、保留扇区及磁盘健康状况等信息进行管理，在磁盘健康状态不满足标准时开启数据迁移，同时该模块还需要提供相关配置接口，以提供配置及状态查询等功能，总结起来该模块可以分为六个功能模块：磁盘 I/O 过滤器、数据迁移、磁盘自愈、磁盘属性管理、磁盘状

态检测及终端配置接口，模块划分图如**错误!未找到引用源。**所示。

不应该出现上面的错误。

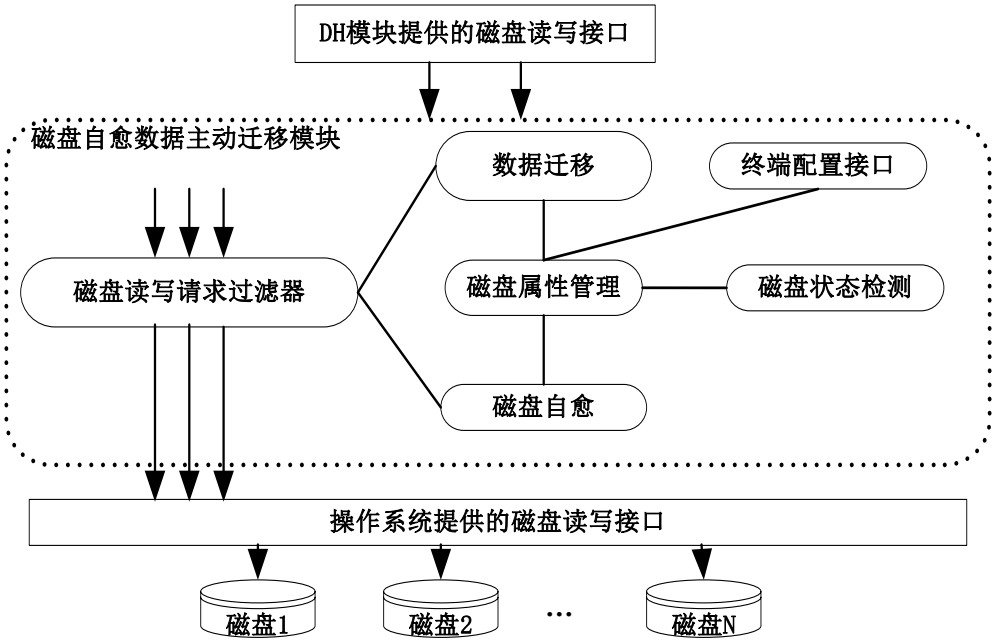

图 3-4 磁盘自愈与数据主动迁移功能模块划分图

磁盘 I/O 过滤器用来过滤磁盘的读写请求，它由多个子过滤器组成，当上层发来磁盘读写请求时，各个子过滤器都会被调用。磁盘 I/O 过滤器会提供注册接口，用来安装子过滤器。在中，磁盘自愈及数据迁移功能模块都会调用注册接口安装子过滤器。磁盘自愈功能模块的子过滤器通过截获磁盘 I/O，分析读写请求的执行状态，将这些信息报告给磁盘属性管理功能模块，如果请求失败了，说明磁盘出现了故障，这时磁盘属性管理会记录相关信息，并调用自愈模块修复故障并将故障扇区数据恢复，同时磁盘属性管理模块根据健康状态属性决定是否需要启动数据迁移来提高数据可靠性。数据迁移功能模块子过滤器截获磁盘 I/O，当其请求的磁盘正在进行数据迁移时，则可能需要将请求重定位到数据迁移的目标磁盘。

3.3 功能模块设计

磁盘自愈与数据主动迁移模块可以划分为磁盘 I/O 过滤器、磁盘属性管理、磁盘自愈、数据迁移、磁盘状态检测及配置接口六个功能模块。磁盘 I/O 过滤器模块分层过滤磁盘读写请求，并分析读写请求执行情况，然后调用其它功能模块

进行相应处理，当磁盘请求处理失败时，会调用磁盘自愈功能模块，当磁盘健康状况不满足标准时，则会调用数据迁移功能模块，因此，磁盘 I/O 过滤器功能模块在磁盘自愈与数据主动迁移模块中非常重要。

3.3.1 I/O 过滤器

读写过滤器是磁盘自愈与数据主动迁移模块的驱动器，同时也是整个数据流动的入口，该功能模块的设计对于整个模块的可移植性非常关键。在设计中使用了分层次过滤磁盘 I/O 的思想，该功能模块提供过滤框架及注册接口，其它功能模块可调用该功能模块提供的注册接口安装过滤器，过滤磁盘 I/O。在磁盘 I/O 过滤器的上层，提供读写磁盘的接口以供 RAID 系统其它模块调用，而在过滤器的最底层，则会调用操作系统提供的磁盘读写接口，真正读写磁盘。

3.3.2 属性管理

磁盘属性管理是整个磁盘自愈与数据迁移模块的核心功能模块，它管理磁盘的健康状况信息，并提供接口供磁盘 I/O 过滤器模块调用以报告对磁盘读写的操作结果。磁盘属性管理模块会维护磁盘健康状态信息，当信息发生改变时，会将相关信息存储到磁盘特定位置，以防止磁盘健康状况属性的丢失。状态信息反馈包括磁盘健康状况、磁盘自愈工作情况以及数据迁移进度等。**错误!未找到引用源。**描述了磁盘属性管理功能模块的划分图。

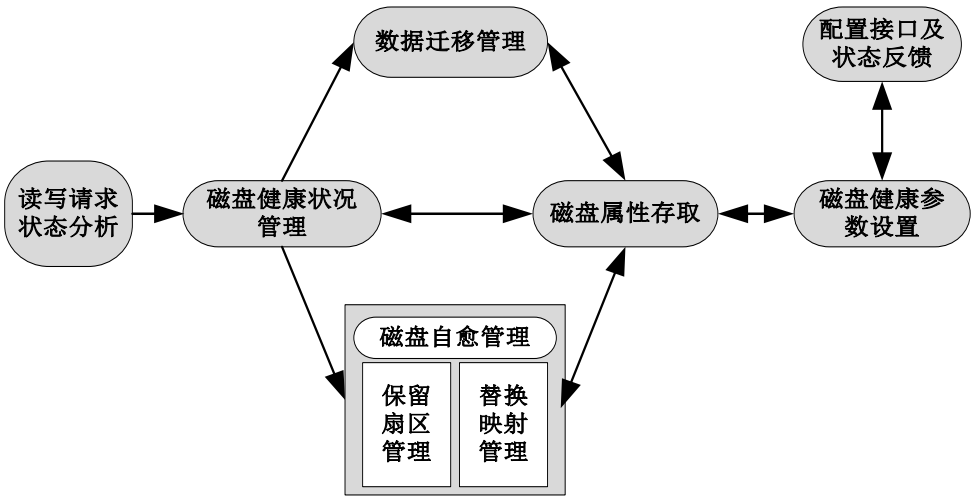

图 3-5 磁盘属性管理功能模块划分

读写请求分析：提供接口供磁盘 I/O 过滤器调用，分析磁盘读写 I/O 执行情况，并将相关信息向磁盘健康管理部分报告。数据迁移管理：管理数据迁移部分信息，提供启动及停止数据迁移相关接口，对数据迁移源磁盘及迁移目标磁盘信息进行维护，根据迁移进度实现目标磁盘替换源磁盘。磁盘自愈管理：管理自愈相关信息，包括保留扇区以及替换映射信息的管理，保留扇区是每个磁盘都保留一部分扇区用来替换磁盘坏扇区。对保留扇区的管理是提供统一的分配保留扇区的接口，供磁盘自愈功能模块调用以实现坏扇区的替换。替换映射信息是使用保留扇区替换坏扇区产生的扇区位置的对应关系，使得以后对坏扇区的读写 I/O 请求会重定位到保留扇区。将磁盘自愈相关信息放入磁盘属性管理中是为了统一对磁盘属性进行管理。

3.3.3 放置策略

在我们实现的原型系统中，定义了几种 Region 类型，分别具有不同的配置和特征（表 3.1）**（表格的交叉引用）**。其中，Type_7 相当于一个 RAM Disk，当然具有比磁盘低的多的响应延迟，可以用来放置那些生存时间非常短（不超过几秒种）的对象，通常是应用程序运行过程中产生的临时文件和同步锁文件。另外三种类型分别对应了三种最常用的 RAID 级别，并具有不同的数据块大小，支持一定范围的应用负载。（所有表必须插入题注，注意在表格的上方，编号形式为 X.X，2 级编号即可，不要 3 级编号，表格格式参照如下，原则上表不能跨页）

表 3-1 原型系统实现的几种类型的 Region 配置

	Type 0	Type 1	Type 5	Type 7
RAID 级别	0	10	5	N/A
磁盘个数	8	8	8	N/A
分条单元大	32KB	16KB	32KB	N/A
数据块大小	512KB	16KB	64KB	1KB
数据块个数	8K	8K	8K	8K

表格由表号、表名、表头、表身等组成。表号按章编，如第 2 章的表为表 2.1、表 2.2、…，第 3 章的表为表 3.1、表 3.2、…等。表名是表格的名称，扼要概括表的内容，字数不宜太多。**表号、表名放在表的正上方**，相对于表体居中排版。

表号及表名后不加标点。表头包括栏头、行头，与表身一起构成表格的主体。表中的竖称为栏，横格称为行。表身的内容，一般包括：数据、文字、公式和表图等。表内的数据对应位要对齐。少数表有表注，表注写在表下面且字号应比表号、表名的字小一号。

所有表格均应在正文中予以引用。引用某表格时，一般写为“…见表 x.y”或“表 x.y 是…”。表格应尽量靠近正文的叙述，一般应先见文，后见表，表不跨节。表格允许转页。表格转页部分可以不写表号和表名，但要重复书写表头，并在表头右上角写“（续）”字标注。

在上一章我们曾经定义了一个“用户指定负载特征”扩展属性页（Page_8000h）。该属性主要用于 OSD 的创建对象（CREATE）命令。利用其属性#1，用户可以直接指定该对象放置在什么类型的 Region，这个属性主要是为了留给我们自己做测试使用，因为一般用户不需要了解 OSD 内部对象放置策略的细节。

3.4 本章小结

本章详细描述了 RAID 系统中磁盘自愈与数据主动迁移模块的设计，对其主要功能模块进行了详细的设计说明。并就各功能模块设计时需要考虑的问题及解决方案进行了讨论。

4 XXX 系统实现

我们将设计实现一个原型系统。该原型系统在传统的（SBC）磁盘阵列控制器的基础上，根据 SCSI OSD 标准实现了新的访问接口，从而改造为一款 OSD 磁盘阵列。支持 OSD 协议的同时，我们还将改变 RAID 的数据组织，设计新的磁盘布局，使之拥有在负载属性控制之下根据数据放置策略优化数据分布的能力。

4.1 过滤器实现

4.1.1 过滤器注册数据结构

```
typedef struct _DEVICE_T{
    ADD_DEVICE_FUN AddDeviceFun; //过滤器增加设备通知函数
    DELETE_DEVICE_FUN  DeleteDeviceFun;  //删除设备
    DELETE_DEVICE_FUN  PrevDeleteDeviceFun; //预删除设备
    DELETE_DEVICE_FUN CanDeleteDeviceFun; //设备是否可删除

struct _list_t{      //各层过滤器连接
            void *next;    //该层过滤器的下层过滤器
void *prev;   //该层过滤器的上层过滤器
    }list_entry;
    char MyData[1]; //该层过滤器私有数据
}DEVICE_T;
```

(论文中出现代码时请遵循以上代码格式，灰色底纹，斜体字，注意论文中不能出现过多的代码，只能出现一些关键数据结构。系统实现过程尽可能的用程序流程图表示。)数据结构中比较重要的字段是 IRP_PROCESS_FUN FunArray，它是过滤读写请求的接口集合，是一组函数指针，指向该功能模块提供的过滤读写请求的接口，这些接口在读写请求到来时会被调用。其它字段主要用于注册设备管理使用，例如 ADD_DEVICE_FUN AddDeviceFun，各功能模块注册

的过滤器不仅过滤读写请求，而且需要得到磁盘删除增加方面的通知(这一部分为了磁盘管理而使用)。list_entry 是用于将各层注册的过滤器连接起来的数据结构。

4.1.2 过滤器工作流程

加入磁盘 I/O 过滤器以后，DH 模块提供给 RAID 系统使用的读写磁盘接口将 RAID 系统发送的磁盘 I/O 请求导入顶层过滤器，即 g_pTopDevice，调用顶层过滤器提供的处理读写 I/O 请求的接口，这些接口对请求进行处理后会调用它下层的过滤器将请求继续向下传递，这样每层过滤器就都能够过滤到读写请求。到达过滤器链表中的最底层就直接调用 SCSI 中间层提供的读写磁盘接口。当读写 I/O 请求返回时，会从最底层向最上层传递，这样各层既能够过滤请求，也能够分析该请求处理的结果。其流程如**错误!未找到引用源。**所示。

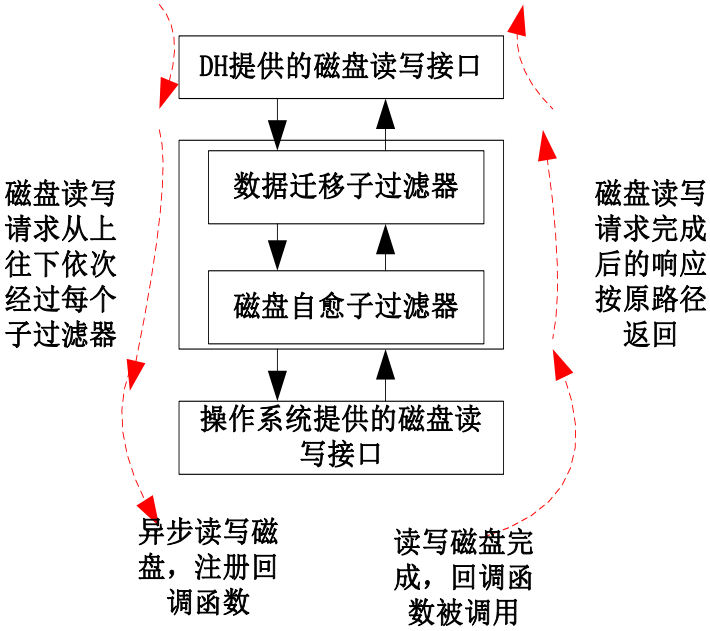

图 4-1 磁盘 I/O 流动

4.2 属性管理模块实现

磁盘属性管理包括磁盘健康状况统计、数据迁移信息的管理、磁盘自愈信息管理及与配置终端接口功能模块交互等功能。磁盘健康状况统计是提供相关

接口供其它功能模块调用，以更新磁盘健康状况信息。数据迁移信息的管理，是指对数据迁移中的源磁盘、目标磁盘、迁移进度及迁移状态位图等信息进行管理(对这些信息的详细说明在数据迁移功能模块实现细节中)。磁盘自愈信息管理，是对保留扇区的管理以及映射表信息的管理。

4.2.1 属性管理数据结构

```
typedef struct _DH_DISK_INFO_T
{    int uMagic;
     DH_TWINS_DISK_T stTwinsDiskInfo;   //数据迁移管理信息
     DH_DISK_CONFIG_T stDiskConfigInfo; //用户配置信息
     DH_DISK_HEALTH_INFO_T stDiskHealthInfo; //磁盘健康相关信息
     DH_SECTOR_INFO_T stSectorInfo; //磁盘预留扇区相关信息
     MEM_POOL_ROOT_T stSectorMapTable;//坏扇区预留扇区映射表内存
     PDH_SECTOR_MAP_T pstHashLink[HASH_TAB_LEN]; //替换映射表
}DH_DISK_INFO_T;
```

磁盘属性管理数据结构包括 uMagic(标识，由于属性信息需要存储到磁盘，因此需要标识该数据项是否有效)、数据迁移管理信息(迁移源磁盘、目标磁盘、迁移进度及迁移状态位图)、磁盘健康相关信息、预留区管理及映射表相关信息。磁盘健康相关信息的更新目前主要依靠 I/O 出错次数以及磁盘状态检测模块通过分析磁盘 SMART 信息得出的预测结果来决定。

4.2.2 属性管理模块工作原理

磁盘属性管理功能模块负责资源管理，提供一系列接口供外部模块调用。该模块在系统启动时，从磁盘相应的位置将磁盘属性读出来，并初始化好替换映射表等信息。在磁盘属性改变时，会将磁盘属性保存到磁盘中。

4.3 数据迁移模块实现

4.3.1 数据迁移相关数据结构

```
typedef struct _DH_TWINS_DISK_T
{   int uMagic;      //标识
```

```
long long uSourceDiskSerial;    //源盘序列号
long long   uDestDiskSerial;    //目标盘序列号
}DH_TWINS_DISK_T;
```

在数据迁移时，使用上述数据结构描述迁移源磁盘与目标磁盘信息，uMagic 用来标识是否数据迁移信息有效，uSourceDiskSerial、uDestDiskSerial 分别是源磁盘与目标磁盘的序列号，uSynPercent 代表迁移进度，pBitMap 是迁移状态位图，标明哪些数据块迁移完成。

4.3.2 数据迁移请求处理

数据迁移需要从源磁盘读出一块数据，然后将这块数据写到目标磁盘中，需要考虑到迁移写操作与正常写操作的同步问题。为保证请求处理的同步，DH 模块对将所有请求的处理都放在一个线程中，其它模块需要读写磁盘时，将请求挂到请求处理队列中。给出了代理线程处理数据迁移请求及其它类型请求大体流程。

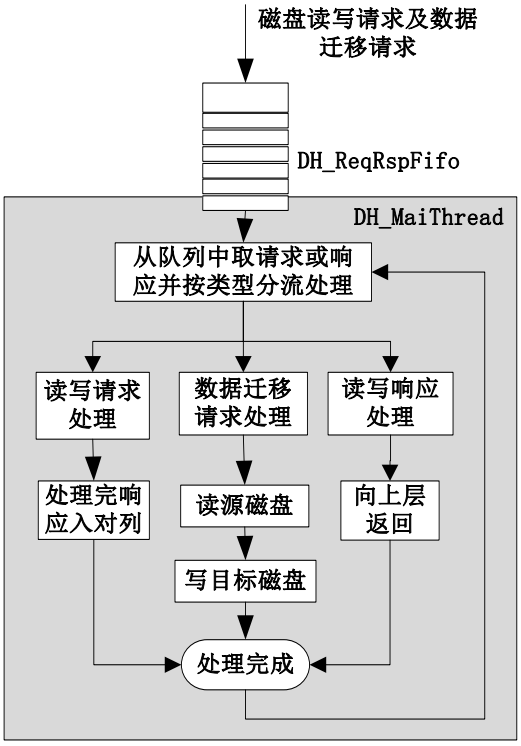

图 4-2 代理线程处理请求及响应

对于数据迁移请求的处理，代理线程 DH_MainThread 线将源磁盘数据读出来，然后再将数据写入到目标磁盘中。在这两步处理中，也有响应加入队列的过程，这部分流程详细执行如下：

（1）使用异步的方式读源磁盘，构造 SCSI 请求并注册回调函数，将请求发给源磁盘；

（2）下层处理完成，调用回调函数通知，回调函数将响应挂回队列；

（3）DH_MainThread 进入下次循环，取出读源磁盘响应；

4.3.3 数据迁移速度调整

降低数据迁移速度是通过在后台迁移线程中增加对迁移队列深度的控制以及增加延迟来实现，该部分控制流程如**错误!未找到引用源。**所示。

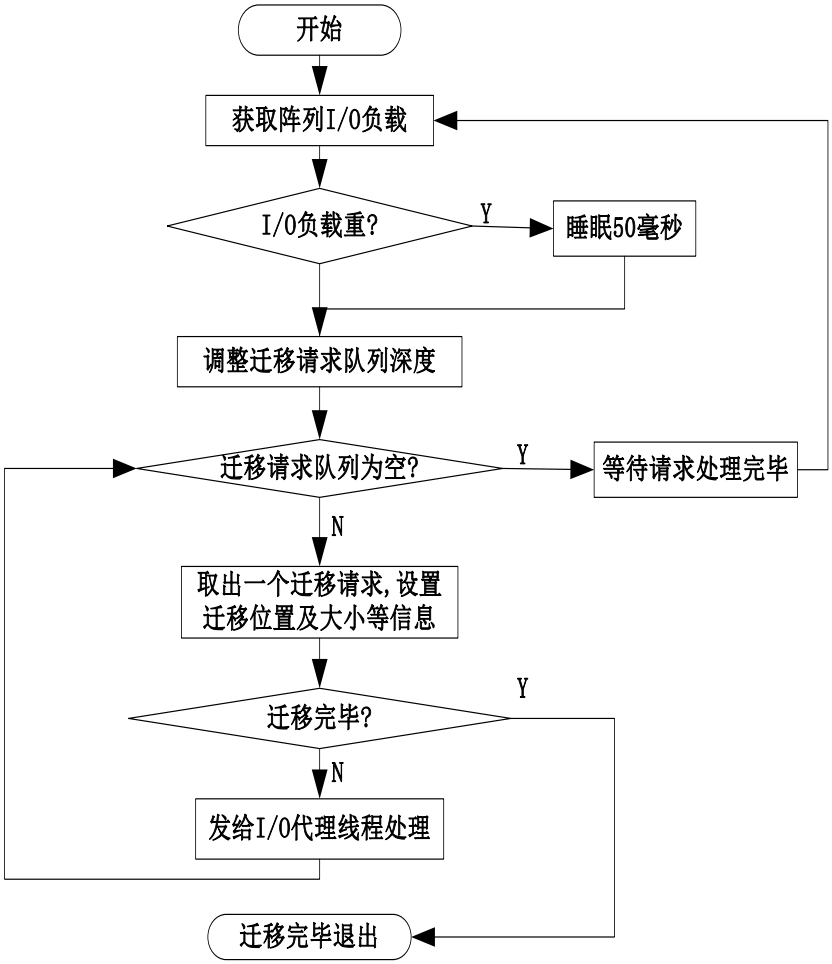

图 4-3 迁移速度控制流程

（标准的流程图一定要有开始和结束当然有些程序没有结束，请注意流程图的画法，Yes/No 分支应该清晰的标注在菱形分支附近，而不是很远，此图中少了一个 N）首先根据检测到的 I/O 负载，判断磁盘阵列当前 I/O 负载，如果 I/O 负载过重(这种情况下通过单一的队列深度控制来调整速度已经无法满足要求)，则每迁移一部分数据就睡眠一段时间(这里设置为 50MS)，然后根据 I/O 负载调整迁移请求队列深度。迁移请求队列是后台迁移线程初始化的用来保存数据迁移请求的队列，通过控制其深度，能够控制后台迁移线程序每次发给 I/O 代理线程的数据迁移请求数目，如果深度较小，那么每次发给 I/O 代理线程的请求数就较小，数据迁移速度就会降低，否则会提高数据迁移速度。

4.4 本章小结

本章描述了磁盘自愈与数据主动迁移模块的具体实现，然后对各功能模块的具体实现进行了详细分析，最后介绍了该模块的动态运行流程。

5 性能测试与分析

5.1 测试环境

我们首先对 RAID 软件进行了评估。测试平台配置为：CPU Intel Xeon 1-8 GHz，512MB 内存，LSI Logic 21320-R (1030) ULTRA320 SCSI HBA，连接 4 块 Seagate Cheetah Ultra320 磁盘（MODEL ST373307LC），一块 Agilent 5221A FC HBA（2Gb），与主机（启动器）的 Qlogic 2310F FC 卡通过光纤连接。为评价磁盘阵列控制器最终性能，我们利用 Iometer 软件对阵列控制器进行了详细的性能评测，具体测试环境见**错误!未找到引用源。**：

表 5-1 光纤通道启动器与阵列控制器配置

配 置	启动器	阵列控制器
CPU	Xeon 2.0	Intel 80321
Memory	512MB	1GB
Fiber Channel HBA	Qlogic 23102G	Agilent DX2
OS	Windows 2003	ARM Linux

5.2 功能测试

5.3 系统界面

不是必须

5.4 性能测试

其中阵列控制器挂接 6 个 Maxtor STM3320820AS 磁盘，利用 6 个磁盘分别构建 RAID0、RAID10、RAID5 三种不同的阵列级别，分条大小为 128K 字

节，缓存大小为 512MByte，LUN 大小为 500GByte，阵列控制器采取写回策略。

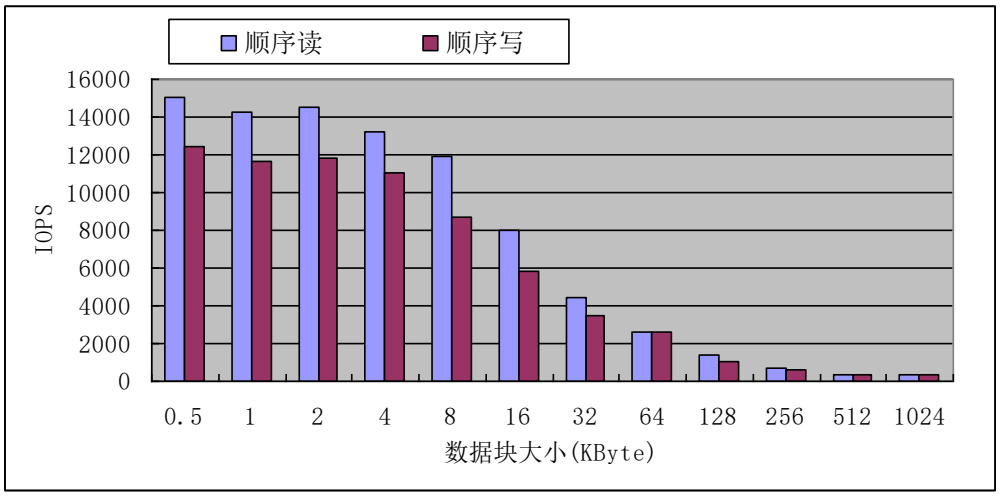

图 5-1 磁盘阵列控制器 RAID5 顺序读写 IOPS 性能

错误!未找到引用源。为 6 个磁盘构建 RAID5 时阵列控制器顺序读写 IOPS(I/O Per Second)性能示意图，从图中可以看出，IOPS 随着数据块大小变大而变小，当数据块大小为 512Byte 时，阵列控制器处理读请求的性能可以达到每秒 15000 个，写性能由于包含冗余校验信息计算，IOPS 略低，约为 12000，此指标高于同类美国阵列 nStor 4502F，其写 IOPS 最大为 7000 左右。

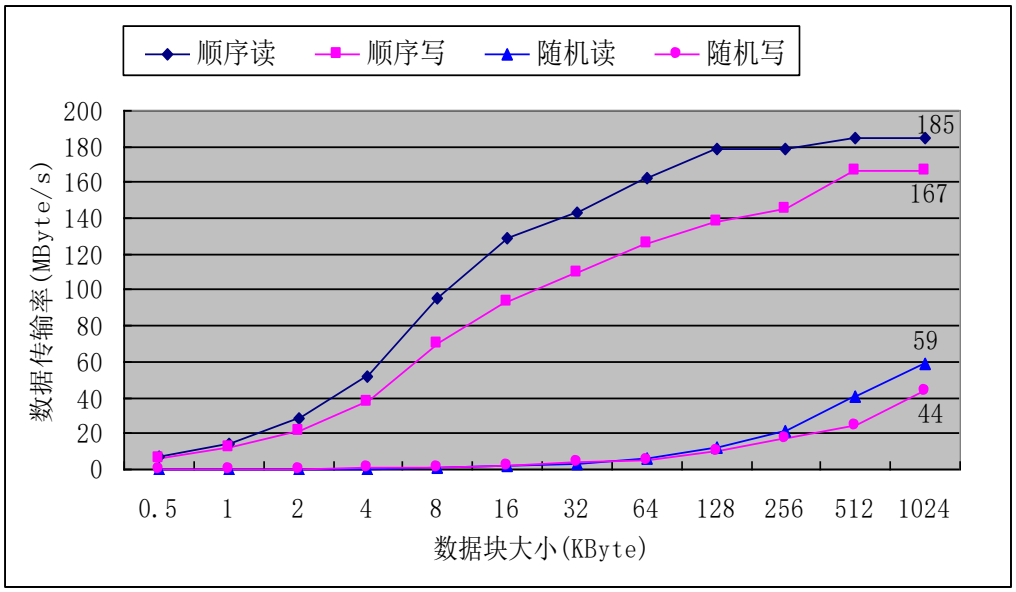

图 5-2 磁盘阵列控制器 RAID5 读写性能

为 6 个磁盘构建 RAID5 时阵列控制器数据传输率示意图，从图中也可以看

出，随着主机端 I/O 请求数据块大小逐渐增大，阵列控制器数据传输率逐渐增加，当数据块大小达到 128Kbyte 时，系统性能最佳，读性能可以达到 185MByte/s，写性能约为 167MByte/s。**（有坐标轴的图必须给出单位，坐标轴意义，所有测试图应该有详细分析）**

从总体性能评测结果看，本文最终设计的 FC-SATA 磁盘阵列控制器基本发挥了 Intel IOP 处理器的最佳性能，同比性能优于同类进口磁盘阵列，基本达到了设计要求，如需要更高的数据传输率，必须同时提升主机通道以及 IOP 本身的性能，才能有实质性的提高。

5.5 本章小结

本章给出了控制器完整的软硬件设计方案，提出了主机板与通道板分离的架构，可以灵活构建全系列阵列控制器产品线；提出了磁盘阵列软件分层异步体系结构，可以有效提高 I/O 请求处理的并行度，减少 I/O 处理同步等待时间；最后提出了一种针对阵列层次性结构的细粒度测试方法，可单独测试评价各模块性能，为系统性能精确评测，以及故障诊断提供决策依据，最后给出了阵列控制器性能综合评测。

6 总结与展望

基于对象的存储是为了克服当前基于块的存储存在的诸多难题，在存储接口和结构层次的重要发展。基于对象的存储设备可以具有更多智能性，可以根据应用负载选择优化的存储策略。实现属性控制的存储策略，关键需要解决两个方面的问题：第一，属性表达什么信息，怎样影响存储策略？第二，属性怎样从用户应用传递给存储设备？围绕这两个问题，作了如下几点研究和工作：

1) 提出了对象属性控制存储策略的模型。OSD 协议作为 SCSI 的扩展集，可以在 TCP/IP 网络和 iSCSI 协议之上来传输，基于对象的存储设备接收 OSD 命令并进行处理。

2) 结合对应用负载特征的分析，提出根据对象属性描述的负载特征动态选择 RAID 级别和分条单元大小的数据放置策略。I/O 请求较小的对象放在 RAID-10 中，同时分条单元尺寸较小；反之则放在 RAID-5 中。

3) 建立了属性的传递机制，一种是应用程序接口静态传递；另一种是动态机制，即根据属性当中包含的文件系统信息预测文件的分类，并根据分类做出进一步的存储策略选择。

4) 论文总结必须用这样的条目的形式给出，总结是对全文的总结，不是对毕业设计的心得，心得部分不要写。

本文为更好地认识基于对象的存储以及 OSD 的实现进行了有益的探索。该领域还有很多研究工作有待完成，包括：

- 以 OSD 为节点构建大规模（PB 级）存储系统
- 属性控制的 caching/prefetching 策略
- 实现 Device-aware（表达设备能力，按需分配资源）
- 设备的学习能力，以文件系统和历史访问信息等为提示进行预测

随着存储需求的爆炸性增长，下一代互联网络必须有新的存储结构来应对构建和管理更大规模存储系统的挑战。基于对象的存储是最有希望的突破点之一。相信存储技术的发展一定能够满足人们不断增长的存储需求。

致 谢

[redacted]

参考文献

[1] Palkar S, Lan C, Han S, et al. E2: a framework for NFV applications[C]. Symposium on Operating Systems Principles. ACM, 2015:121-136.

[2] Bremler-Barr A, Harchol Y, Hay D. OpenBox: A Software-Defined Frame work for Developing, Deploying, and Managing Network Functions[C]. C onference on ACM SIGCOMM 2016 Conference. ACM, 2016:511-524.

[3] Sekar V, Egi N, Ratnasamy S, et al. Design and implementation of a con solidated middlebox architecture[C]. Usenix Conference on Networked Sys tems Design and Implementation. 2011:24-24.

[4] Anderson J W, Braud R, Kapoor R, et al. xOMB: extensible open middle boxes with commodity servers[C]. Eighth ACM/IEEE Symposium on Arch itectures for NETWORKING and Communications Systems. ACM, 2012:4 9-60.

[5] A. Gember, R. Grandl, A. Anand, T. Benson, and A. Akella. Stratos: Virt ual Middleboxes as First-class Entities[R]. Technical report, UW-Madison 2012.

[6] Zhang W, Liu G, Zhang W, et al. OpenNetVM: A Platform for High Per formance Network Service Chains[C]. The Workshop on Hot Topics in M iddleboxes and Network Function Virtualization. ACM, 2016:26-31.

[7] Gember-Jacobson A, Viswanathan R, Prakash C, et al. OpenNF: enabling innovation in network function control[C]. ACM Conference on SIGCOM M. ACM, 2015:163-174.

[8] Rajagopalan S, Dan W, Jamjoom H, et al. Split/merge: system support fo r elastic execution in virtual middleboxes[C]. Usenix Conference on Netw orked Systems Design and Implementation. 2013:227-240.

[9] J. Khalid, A. Gember-Jacobson, R. Michael, A. Abhashkumar, and A. Ak ella. Paving the Way for NFV: Simplifying Middlebox Modifications Usin g StateAlyzr[C]. In Proc. of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI'16), 2016.

[10] Ballani H, Costa P, Gkantsidis C, et al. Enabling End-Host Network Func

tions[J]. ACM SIGCOMM Computer Communication Review, 2015, 45(4): 493-507.

[11] Paxson V. Bro: a system for detecting network intruders in real-time[C]. Conference on Usenix Security Symposium. USENIX Association, 1998:3-3.

[12] Yoan N. Iptables[M]. Miss Press, 2013.

[13] Sherry J, Gao P X, Basu S, et al. Rollback-Recovery for Middleboxes[C]. ACM Conference on Special Interest Group on Data Communication. A CM, 2015:227-240.

[14] Chrome G. HTTP persistent connection[J]. 2015.

[15] Surhone L M, Tennoe M T, Henssonow S F, et al. Ffmpeg[M]. 2010.

[16] Housley R, Hoffman P. Internet X.509 Public Key Infrastructure Operatio nal Protocols: FTP and HTTP[J]. Ietf Rfc Sri Network Information, 1999, 11(3):82--89.

[17] Maheshwari A, Sharma A, Ramamritham K, et al. TranSquid :Transcoding and Caching Proxy for Heterogenous E-Commerce Environments[C]. Inte rnational Workshop on Research Issues in Data Engineering: Engineering E-Commerce/e-Business Systems. IEEE Computer Society, 2002:50.

[18] J. Khalid, A. Gember-Jacobson, R. Michael, A. Abhashkumar, and A. Ak ella. Paving the Way for NFV: Simplifying Middlebox Modifications Usin g StateAlyzr[C]. In Proc. of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI'16), 2016.

[19] NFV Paper [EB/OL].https://portal.etsi.org/NFV/NFV White Paper2.pdf,2017 -05-05.

[20] Hwang J, Ramakrishnan K K, Wood T. NetVM: high performance and fl exible networking using virtualization on commodity platforms[C]. Usenix Conference on Networked Systems Design and Implementation. USENIX Association, 2014:445-458.

[21] S. Han, K. Jang, A. Panda, S. Palkar, D. Han, and S. Ratnasamy. SoftNI C: A Software NIC to Augment Hardware[R]. Technical report, EECS De partment, University of California, Berkeley, 2015..

[22] Martins J, Ahmed M, Raiciu C, et al. ClickOS and the art of network fu

nction virtualization[C]. Usenix Conference on Networked Systems Design and Implementation. USENIX Association, 2014:459-473.

[23] Panda A, Han S, Jang K, et al. NetBricks: taking the V out of NFV[C]. Usenix Conference on Operating Systems Design and Implementation. US ENIX Association, 2016:203-216.

[24] A. Gember, R. Grandl, A. Anand, T. Benson, and A. Akella. Stratos: Virt ual Middleboxes as First-class Entities[R]. Technical report, UW-Madison 2012.

[25] Palkar S, Lan C, Han S, et al. E2: a framework for NFV applications[C]. Symposium on Operating Systems Principles. ACM, 2015:121-136.

[26] Rajagopalan S, Dan W, Jamjoom H. Pico replication:a high availability fr amework for middleboxes[C]. Symposium on Cloud Computing. 2013:1-15.

[27] Sherry J, Gao P X, Basu S, et al. Rollback-Recovery for Middleboxes[C]. ACM Conference on Special Interest Group on Data Communication. A CM, 2015:227-240.

[28] Qazi Z A, Tu C C, Chiang L, et al. SIMPLE-fying middlebox policy enf orcement using SDN[J]. Computer Communication Review, 2013, 43(4):27 -38.

[29] Zhang W, Hwang J, Rajagopalan S, et al. Flurries: Countless Fine-Graine d NFs for Flexible Per-Flow Customization[C]. International on Conferenc e on Emerging NETWORKING Experiments and Technologies. ACM, 201 6:3-17.

[30] Bremler-Barr A, Harchol Y, Hay D. OpenBox: A Software-Defined Frame work for Developing, Deploying, and Managing Network Functions[C]. C onference on ACM SIGCOMM 2016 Conference. ACM, 2016:511-524